BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 1

Roll no: 42 PRN No: 12414728 Name of Student: **Saachi Laturiya**

Subject Name : Deep Learning

Problem Statement : Install and configure TensorFlow/Keras in Google Colab. Load the KMNIST dataset, perform data preprocessing, normalization, train-test split verification, and visualize sample images.

Algorithm :

1. Open Google Colab and install the required Python packages.
2. Import TensorFlow, NumPy, Matplotlib, and the KMNIST dataset.
3. Load the KMNIST dataset into training and testing sets.
4. Display the dimensions and basic information of the dataset.
5. Visualize a sample KMNIST image.
6. Display multiple sample images for dataset exploration.
7. Convert image data type from uint8 to float32.
8. Normalize pixel values from **0–255** to **0–1**.
9. Verify the normalization by checking the minimum and maximum pixel values.
10. Display a normalized sample image.
11. Verify the training and testing sample distribution.

Code :

1. Import the required libraries for deep learning, numerical computation, visualization, and loading the KMNIST dataset.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

from extra_keras_datasets import kmnist
```

2. Load the KMNIST dataset into training and testing sets.

```
(X_train, y_train), (X_test, y_test) = kmnist.load_data()
```

3. Display the dimensions of the training and testing datasets.

```
print("Training Images :", X_train.shape)
print("Training Labels :", y_train.shape)

print("Testing Images  :", X_test.shape)
print("Testing Labels  :", y_test.shape)
```

4. Display the image size, number of classes, and pixel value range of the dataset.

```
print("Image Size :", X_train.shape[1], "x", X_train.shape[2])
print("Number of Classes :", len(np.unique(y_train)))
print("Pixel Value Range :", X_train.min(), "to", X_train.max())
```

5. Visualize a sample KMNIST image from the training dataset.

```
plt.figure(figsize=(4,4))
plt.imshow(X_train[0], cmap='gray')
plt.title(f"Label: {y_train[0]}")
plt.axis('off')
plt.show()
```

6. Display multiple sample images to understand the dataset distribution.

```
plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(f"Label: {y_train[i]}")
    plt.axis('off')

plt.tight_layout()
plt.show()
```

7. Convert the image data type from uint8 to float32 for preprocessing.

```
print("Before Conversion:", X_train.dtype)

X_train = X_train.astype('float32')
X_test = X_test.astype('float32')

print("After Conversion :", X_train.dtype)
```

8. Normalize the pixel values from the range 0–255 to 0–1.

Verify that the pixel values have been normalized successfully.

```
X_train = X_train / 255.0
X_test = X_test / 255.0

print("Minimum Pixel Value:", X_train.min())
print("Maximum Pixel Value:", X_train.max())
```

9. Display a normalized sample KMNIST image after preprocessing.

```
plt.figure(figsize=(4,4))
plt.imshow(X_train[0], cmap='gray')
plt.title("Normalized KMNIST Image")
plt.axis('off')
plt.show()
```

Output :

```
Training Images : (60000, 28, 28)
Training Labels : (60000,)
Testing Images  : (10000, 28, 28)
Testing Labels  : (10000,)
```

```
Image Size : 28 x 28
Number of Classes : 10
Pixel Value Range : 0 to 255
```

Figure 1: Sample KMNIST character image from the training dataset.

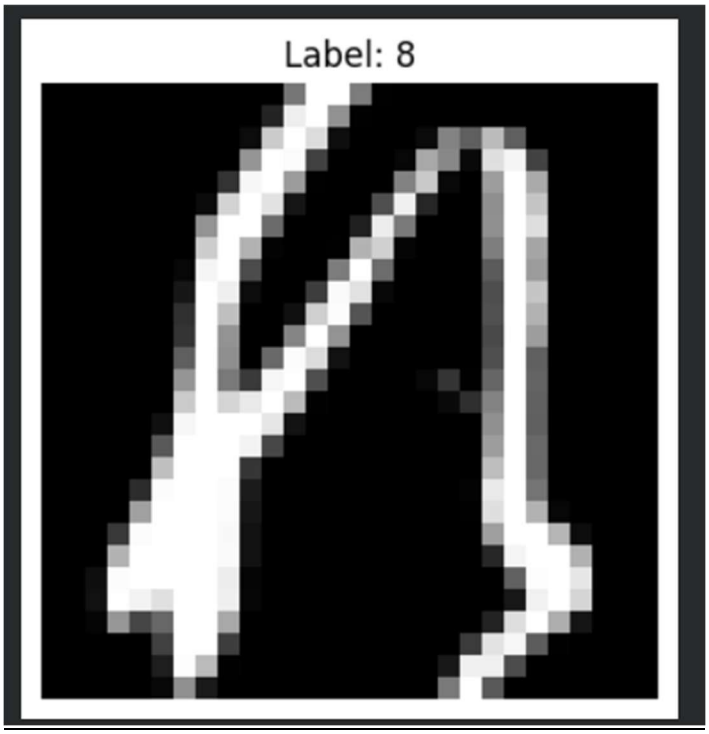
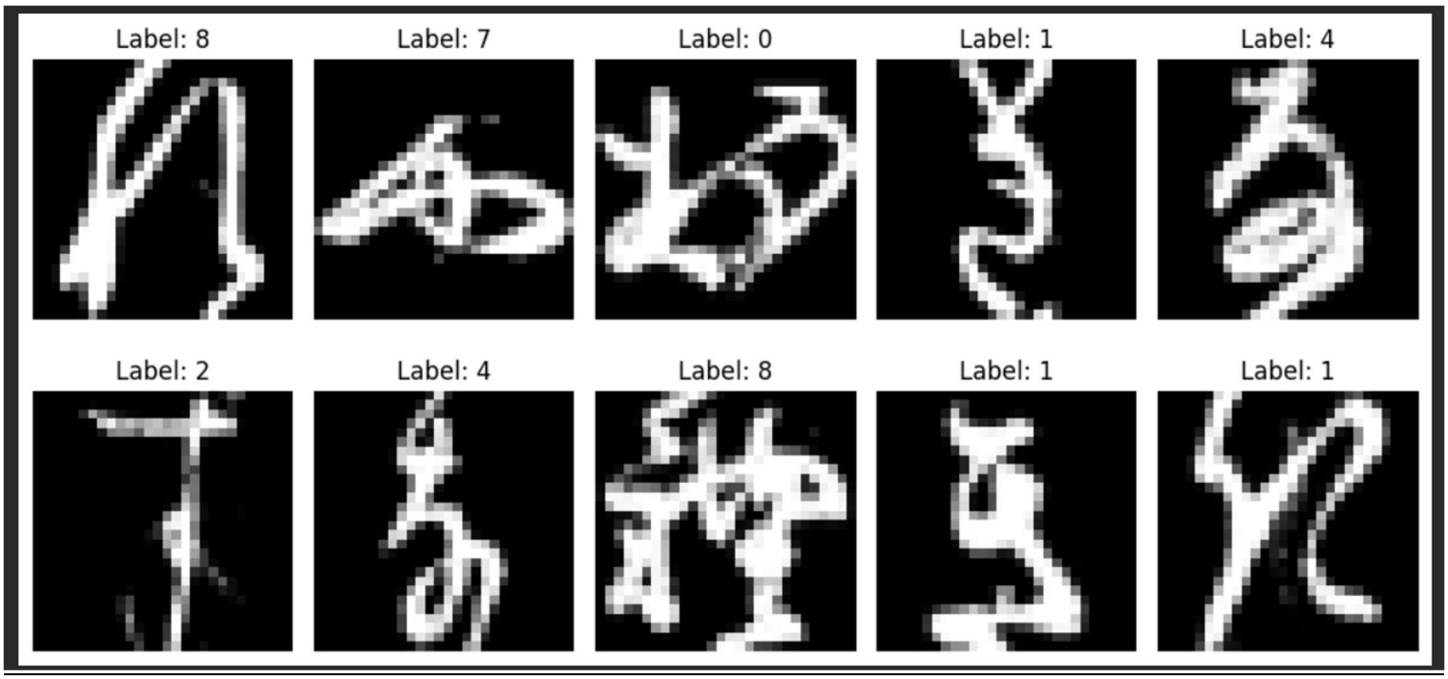


Figure 2: Visualization of the first ten sample KMNIST characters from the training dataset.




```
Before Conversion: uint8
After Conversion : float32
```

```
Minimum Pixel Value: 0.0
Maximum Pixel Value: 1.0
```

Figure 3: Normalized sample KMNIST character after preprocessing.

